

What this project does — a guide for the team

EEE 402 · Group 10 · github.com/AlAminAshraf01/speaker-count-separate-v1 · printed 24 Sep 2026

Read this first. It takes about five minutes and assumes you have not seen any of the code.

The problem

Give the system **3 seconds of audio in which several people are all talking at the same time**. It has to do two things:

1. **Count** how many people are talking (between 1 and 5).
2. **Separate** them — output one clean audio track per person.

Everyone talks for the whole 3 seconds. Nobody takes turns. That makes it harder than it sounds, and it is worth knowing that this is *not* the same problem as figuring out who speaks when in a normal conversation (that is called diarisation, and it is a different task).

Where the audio comes from

LibriSpeech, a public dataset of people reading audiobooks, packaged as **LibriMix**. We take single-person recordings and add them together ourselves to build mixtures, so we always know the true answer.

The one rule that matters: **the people in the test set never appear in training**. 201 speakers for training, 32 for tuning, 32 for the final test, with zero overlap. If a speaker appeared in both, the system could recognise voices instead of counting them and our scores would be fiction.

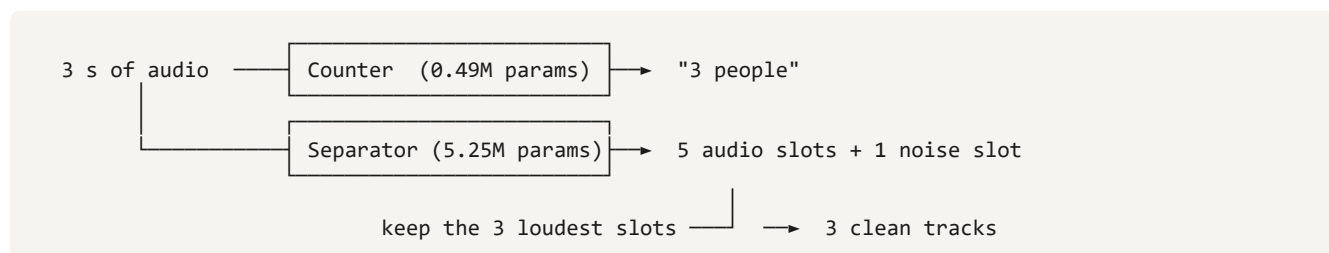
We also add background noise (white/pink/brown) to 75% of clips and vary the volume slightly, so the system cannot cheat by listening for "how loud is it".

The design: two models, not one

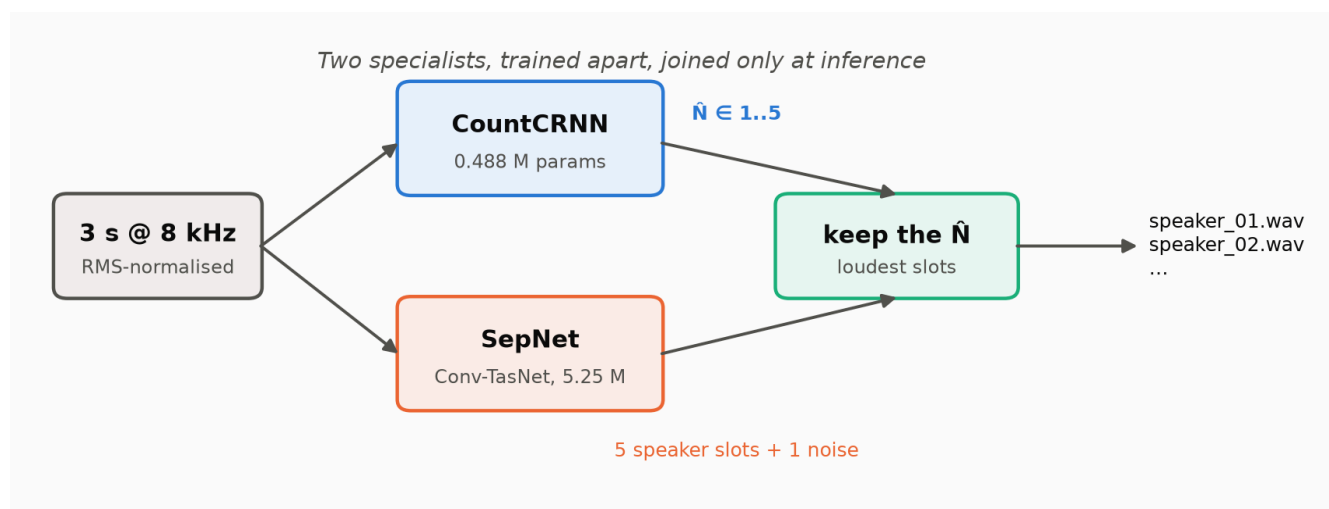
This is the single most important thing to understand, and it is the reason this version exists.

The **previous version used one network to do both jobs**. It did neither well. When we measured where the training signal actually went, the counting half was receiving **0.53%** of it and the separation half was taking **86%** — a ratio of about **633 to 1**. The counting half never learned anything. It ended up answering "**1 speaker**" to almost every clip, which scores **20%** on a 5-class problem. That is exactly what you would get by guessing.

So this version uses two separate models, trained independently and only joined at the end:



Each model now gets 100% of its own training signal. It costs about 5% more computing power. Counting went from **20% to 91%**.



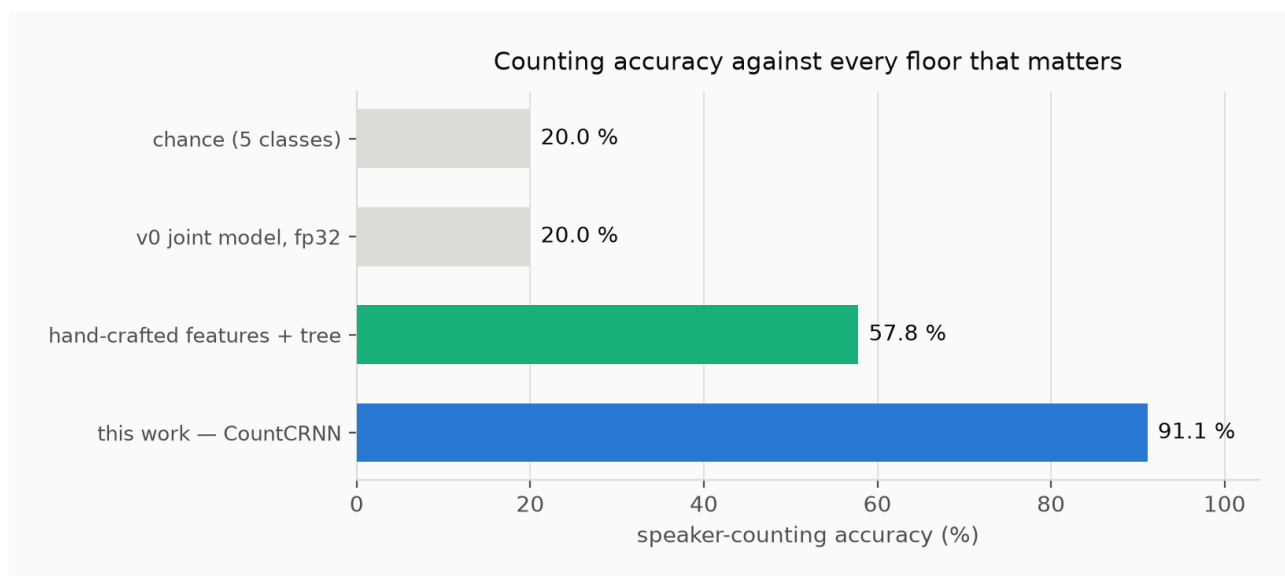
The results

Measured once, at the end, on 1,500 mixtures the models had never seen, from 32 speakers who were never in training.

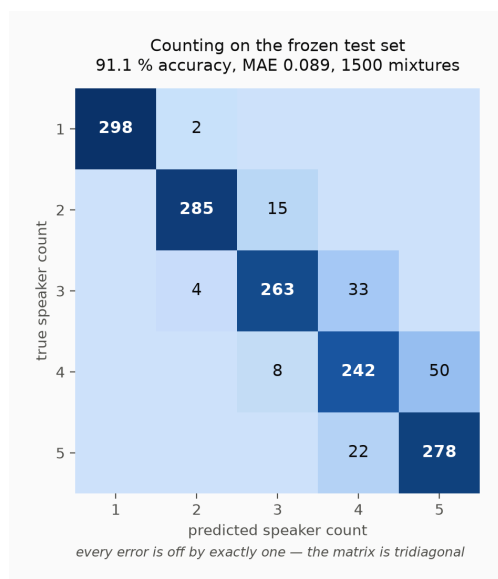
Counting: 91.1% correct

Our system	91.1%
Guessing	20.0%
The old single-network version	20.0%
A simple decision tree on hand-made measurements	57.8%

The best part is not the 91%. It is that **every single mistake is off by exactly one**. When it is wrong, it says 3 when the answer is 4 — never 2, never 5. Out of 1,500 clips there is not one error bigger than a neighbour. The average error size is **0.089 speakers**, against 2.000 for the old version.



The grid below shows every one of the 1,500 test clips. Read a row as "the true answer was this" and a column as "the system said this". Everything sits on the diagonal or right next to it — nothing lands two squares away.

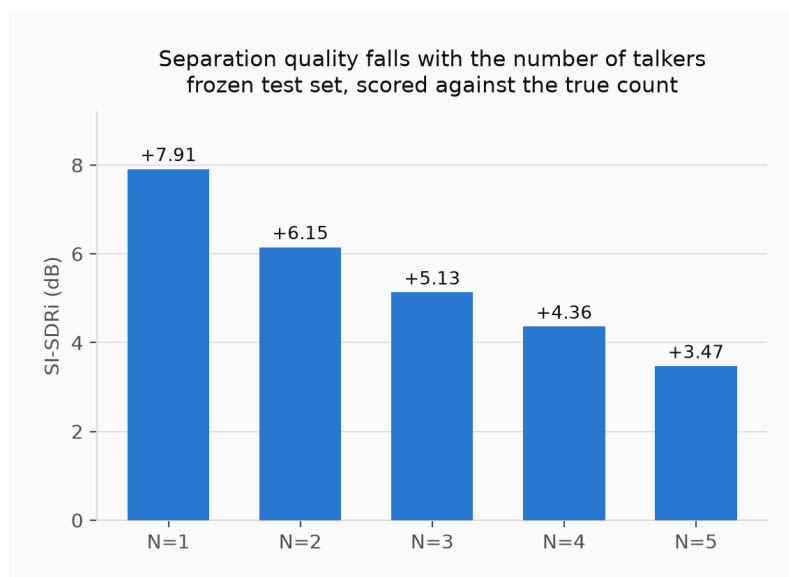


Separation: +4.63 dB improvement

"dB improvement" means: how much cleaner each person's voice is after separation than it was in the mixture. Higher is better; 0 would mean we achieved nothing.

people talking	improvement
1	+7.9 dB
2	+6.2 dB
3	+5.1 dB
4	+4.4 dB
5	+3.5 dB

More people = harder = smaller improvement. That is expected and it is the right shape.



Important for the report: published research reports 14.76 dB, and we should *not* claim we fell short of it. That number is for a model that is **told** there are exactly 2 speakers, trained **200 epochs** on **six times more audio**. Ours handles 1–5 speakers, trained 30 epochs, on a free Kaggle account. Three different things — say so rather than apologising for a gap.

The most interesting thing we found

Partway through, we discovered a bug in how training data was being fed to the models. Each epoch was supposed to generate **fresh** mixtures. Because of one wrong setting, it was silently re-using **the same mixtures every single epoch**.

We fixed it and re-trained both models. The result was not what we expected:

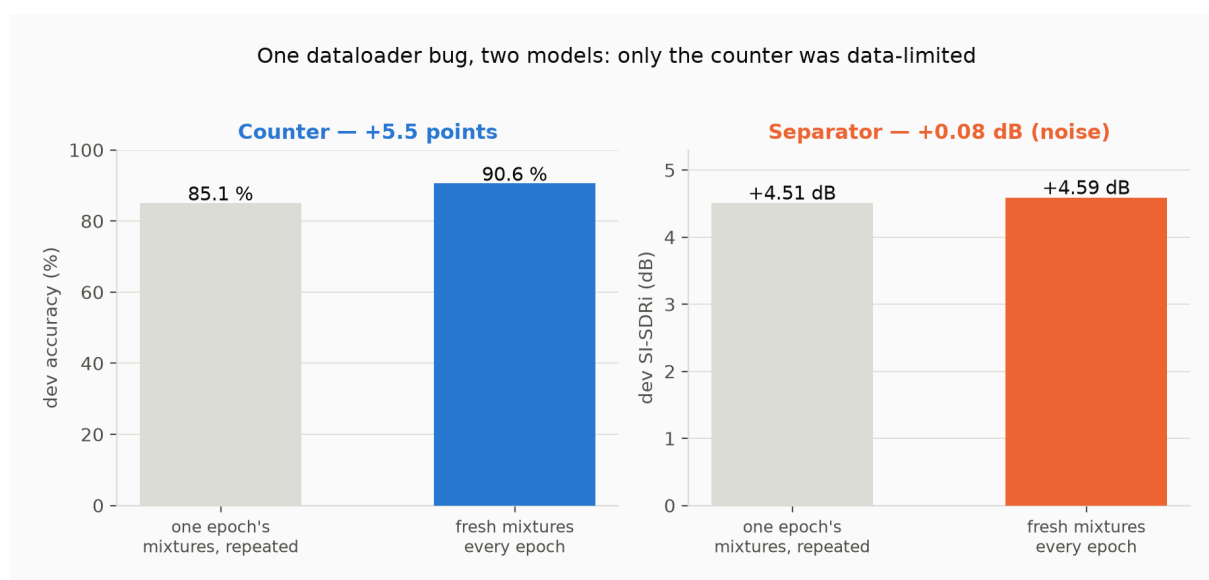
	before the fix	after the fix
Counter	85.1%	90.6% (+5.5 points)
Separator	+4.51 dB	+4.59 dB (+0.08 dB — nothing)

Same bug, same data, same computing time — and it mattered enormously for one model and not at all for the other.

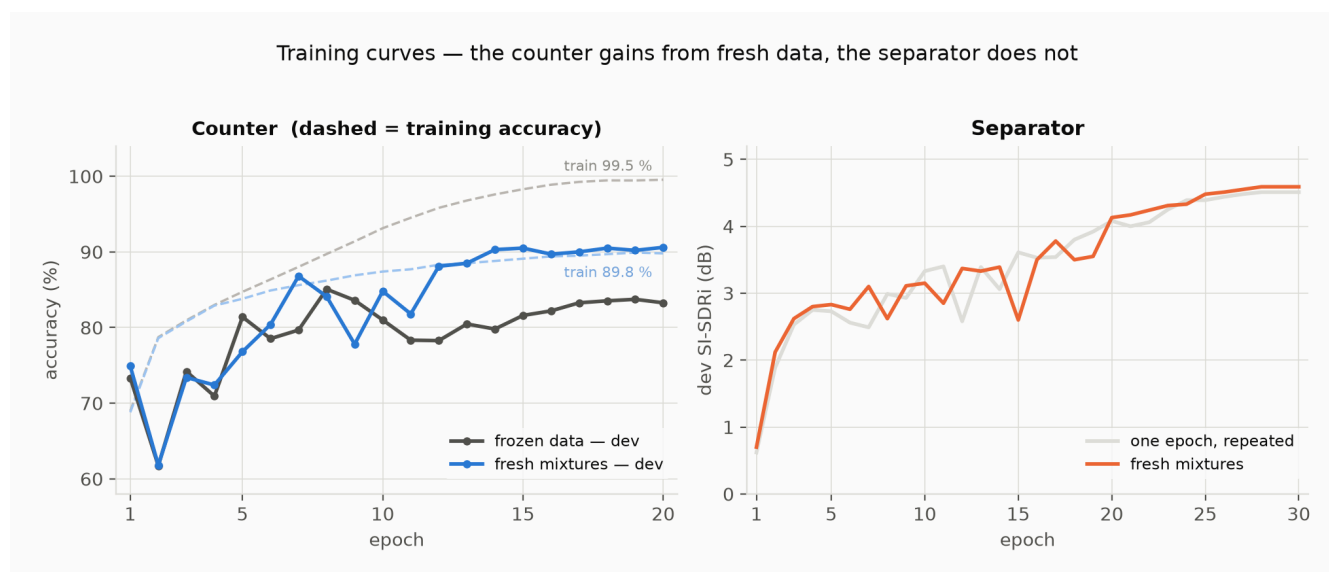
The reason: the counter is small and the task is simple enough that it had **memorised** its fixed set of examples. You can see it in the training curves — its training score climbed to 99.5% while its real score stalled at 83%. That gap is the memorisation. After the fix the gap disappeared entirely.

The separator is ten times bigger but its job is to reconstruct audio waveforms, which it cannot memorise from 12,000 examples. It was never limited by data — it is limited by training time.

This is a genuinely good experimental result: one variable changed, two models, opposite outcomes, and a measurable explanation for why. **It is worth a section in the report.**

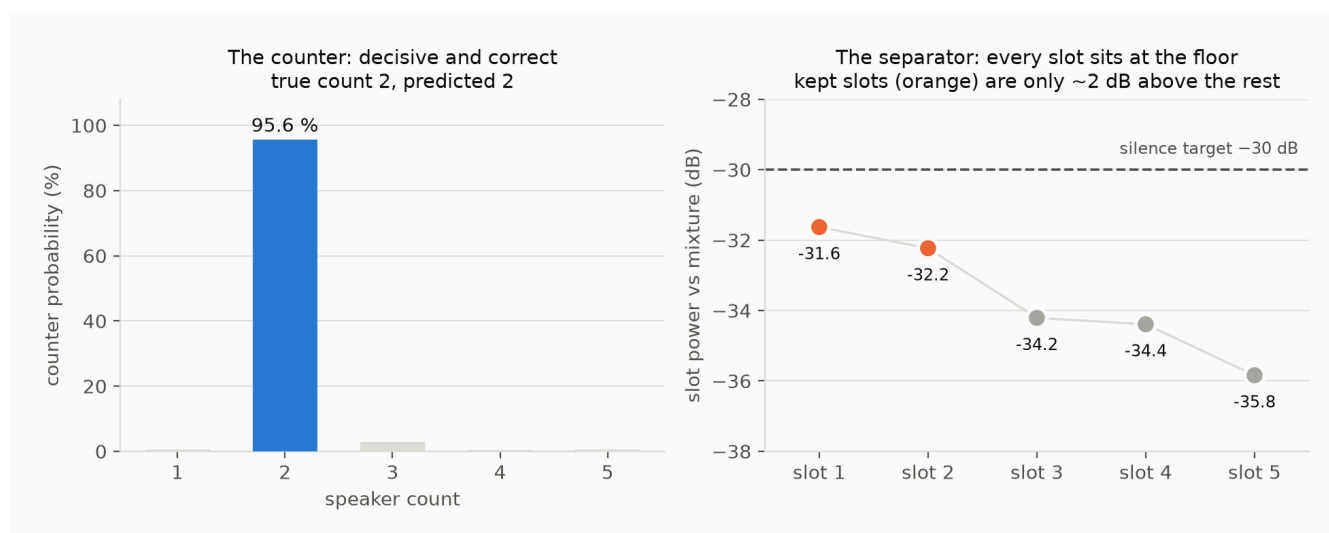


The training curves show *why*. In the left panel the dashed lines are training accuracy. For the first eight epochs the two runs are identical. Then the broken run (grey) keeps climbing on its training data — all the way to 99.5% — while its real score stalls. It is memorising answers instead of learning to count. The fixed run (blue) never does that: its training and real scores stay together. On the right, the separator's two runs lie almost on top of each other, which is what "this model was never short of data" looks like.



What it looks like on a single clip

Notebook 05 runs the whole system on one clip and lets you listen. This one is from the test set, with two people talking:



Left — the counter. It puts 95.6% of its confidence on "2", and 2 is correct. That is what a model that genuinely knows looks like: one tall bar, not a spread.

Right — something we did not expect. The separator always produces 5 audio slots, and we keep the loudest N. The design assumed the real speakers would be loud and the spare slots would be pushed down to near-silence, leaving an obvious gap. Instead, **every slot is near-silent** — all five sit below the -30 dB silence line — and the two real speakers are only about 2 dB louder than the spares.

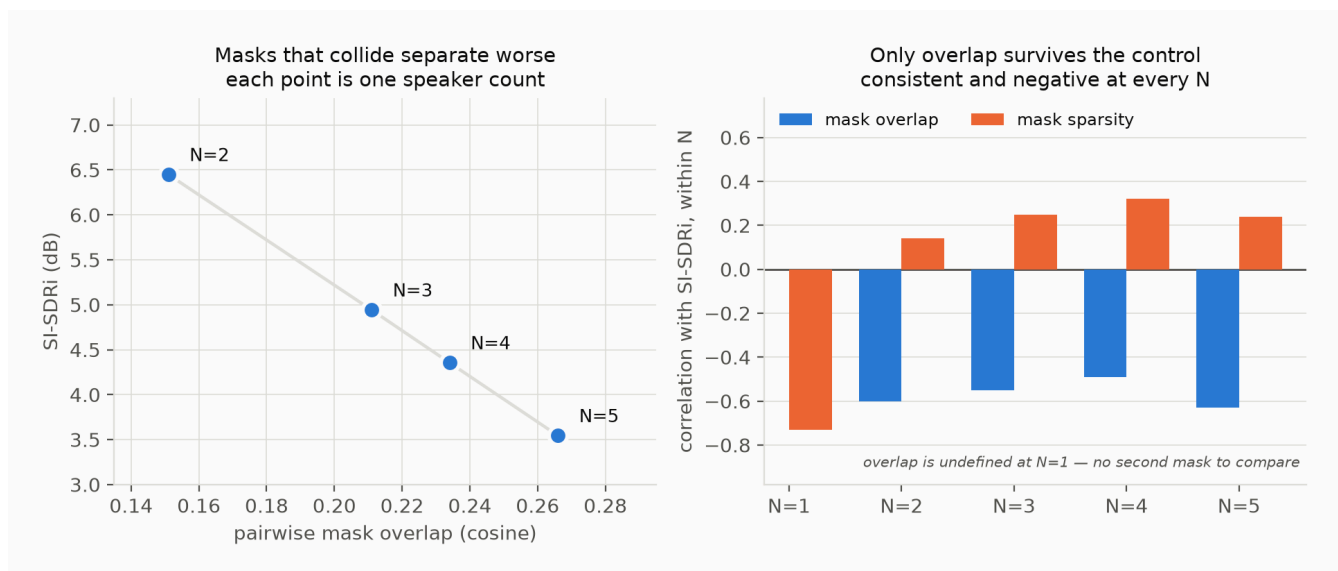
The reason is a property of the scoring method. Separation quality is measured with **SI-SDR**, which is deliberately *scale-invariant*: it judges the *shape* of a sound wave and ignores how loud it is. Since the model is trained to maximise that score, nothing ever tells it to get the volume right — and a separate part of training pushes the spare slots quieter. With a push in one direction and nothing pushing back, everything drifts quiet.

What this means in practice:

- **The accuracy numbers are still correct.** They are scale-invariant too, so they measure whether each voice was separated properly, which it was.
- **The raw audio comes out very quiet**, about 30 dB below the input. The demo now boosts the separated tracks by a single shared amount so you can hear them, and prints exactly how much.
- **Choosing which slots are real is working on a thin margin** — 2 dB rather than a cliff. It still works (the counter-sensitive score is positive), but it is worth one sentence in the report, and it points at an obvious improvement: add a loss term that cares about volume.

Looking inside the separator

Why does separation get worse with more people? We measured how much the separator's internal "masks" — one per speaker — overlap with each other. More overlap means two speakers are fighting over the same parts of the sound.



Left: as more people talk, the masks overlap more, and separation gets worse — a clean straight-line relationship.

Right: the careful part. Both overlap and quality change with the number of speakers, so a correlation between them could be a coincidence of that. So we checked *within* each speaker count separately. Overlap still predicts quality every time (blue bars, all negative). The other measure we tried, "sparsity", does not — it flips direction once you control for the number of speakers (orange). We report both.

How to run it

Everything runs on a **free Kaggle account**. Six notebooks, in order. Full instructions with screenshots-level detail are in `docs/KAGGLE_RUNBOOK.md`.

#	notebook	needs a GPU?	time
00	build the dataset	no	15 min
01	check for cheating + simple baseline	no	40 min
02	train the counter	yes	2.5 h
03	train the separator	yes	9 h
04	final scoring	yes	3 min
05	demo — listen to it working	no	2 min

Notebooks 00, 01 and 05 use **no GPU time at all**. Kaggle gives 30 GPU-hours a week; the whole project uses about 11.

Never edit the `.ipynb` files directly. Edit `notebooks/src/*.py`, then run `python tools/build_notebooks.py`. There is an automatic check that refuses to run if you forget.

What is in the repo

<code>notebooks/</code>	the six Kaggle notebooks (upload these)
<code>src/countsep/</code>	the actual model and data code
<code>scripts/</code>	one script per stage, called by the notebooks
<code>docs/</code>	KAGGLE_RUNBOOK.md (how to run), DESIGN.md (why), DATA.md, DIAGNOSIS.md
<code>report/</code>	figures for the report + RESULTS.md with every table
<code>tests/</code>	run <code>`python tools/run_all_tests.py`</code> – 6 files, under a minute

Four things we must be honest about in the report

These are not weaknesses to hide. Writing them down first is what makes the rest credible, and an examiner will find them anyway.

- 1. Everyone talks at once, the whole time.** Real conversations are mostly people taking turns. Our system does not handle that and was never trained for it. Say so.
- 2. The test set is 1,500 clips but only 32 people.** So the clips are not fully independent. We report a confidence interval of **89.5% to 92.4%** and should quote that rather than the bare 91.1%.
- 3. Half of one interpretability claim failed.** We predicted that as more people talk, the separator's internal "masks" would overlap more *and* become less sparse. The overlap part held. The sparsity part did not. We report both. A half-confirmed mechanism honestly reported is worth more than a fully-confirmed one that nobody checked.
- 4. The comparison with published results is not apples-to-apples.** See the separation section.

One-paragraph summary, if someone asks

We built a system that listens to three seconds of overlapping speech, says how many people are talking, and separates their voices. The previous version used a single network for both jobs and the counting half scored at chance — we measured that it was receiving half a percent of the training signal. We split it into two specialist models. Counting went from 20% to 91%, with every remaining error off by only one, and separation improves each voice by about 4.6 dB. Along the way we found a data-pipeline bug that had been silently re-using the same training examples every epoch; fixing it gained the counter 5.5 points and the separator nothing, which told us which of the two was actually short of data.